CustomButton Add-in

Functional Understanding Document

Version 1.1

Table of Contents

1 Introduction 4
1.1 What is CustomButton? 4
1.2 Installation of CustomButton.xlam 4
1.2.1 Downloading the CustomButton Add-in 4
1.2.2 Important: Unblocking the Add-in 5
2 Working with CustomButton 6
2.1 Referencing CustomButton in VBA 6
3 CustomButtons "Quick Start" Guide 7
3.1 ShowUserFormCode() Function 7
3.2 Deleting the Legacy CommandButtons 8
3.3 _Entry and _Exit Events 8
3.4 FindWindow() Windows API 9
3.5 CentreUserForm() Function 9
4 Functional Specifications 9
4.1 Class CustomButtons 9
4.1.1 .Create() Method 10
4.1.2 .DistroyAll() Method 11
4.1.3 .ShowUserFormCode() Method 12
4.1.4 .ExcelGreen Property 12
4.1.5 .Trace Property 13
4.2 Class CustomButton 14
4.2.1 Event Support 14
4.2.2 Methods 14
4.2.3 Properties 14
4.2.4 Style & Color Properties 15

List of Tables

Table 4-1: CustomButtons Class Methods 10
Table 4-2: CustomButtons Class Properties 10
Table 4-3: CustomButtons.Create() Parameters 11
Table 4-4: CustomButtons.DistoryAll() Parameters 12
Table 4-5: CustomButtons.ShowUserFormCode() Parameters 12
Table 4-6: CustomButtons.ExcelGreen Porperty 13
Table 4-7: CustomButtons.Tracen Porperty 13
Table 4-8: CustomButton Methods 14
Table 4-9: CustomButton Methods 15
Table 4-10: Default Color Schemes by Operating System / Application 16

List of Figures

Figure 1-1: Sample Userform with CustomButtons 4
Figure 1-2: Excel Add-ins Folder 5
Figure 1-3: Unblock Downloaded CustomButton Add-in 5
Figure 2-1: Adding CustomButton_v1_4 as a Reference 6
Figure 3-1: ShowUserFormCode() Output 8

1 Introduction

Welcome to **CustomButton**, a modern UI framework for Microsoft Excel.
For decades, VBA developers have been restricted to the "Legacy" aesthetic of standard Windows controls—designs that have remained largely unchanged since the mid-1990s. While functional, these controls often feel out of place in the modern professional landscape of Windows 10 and 11.

CustomButton was developed to bridge this gap.

1.1 What is CustomButton?

CustomButton is a lightweight Microsoft Excel Add-in designed to modernize the legacy **VBA UserForm** interface. CustomButton replicates the clean, minimalist aesthetic of Windows 10 & 11, replacing the dated look of standard UserForm CommandButtons with a contemporary design.

Figure 1-1: Sample Userform with CustomButtons

1.2 Installation of CustomButton.xlam

Follow the steps below to install the **CustomButton** add-in and integrate it with Microsoft Excel. These steps ensure the library is correctly registered and available for use within the VBA Editor.

1.2.1 Downloading the CustomButton Add-in

To install the Add-in, download the **CustomButton v1.4.xlam** file from the GitHub repository and move it to the local Microsoft Add-ins directory:

C:\Users\[*YourUsername*]\AppData\Roaming\Microsoft\AddIns

Note: Replace [YourUsername] with your active Windows login name.

Quick Access Shortcut:
Alternatively, you can access this folder directly by pasting the following environment variable into the File Explorer address bar:

%appdata%\Microsoft\AddIns

Once the folder is open, drag and drop the CustomButton v1.4.xlam file from your **Downloads** folder into the **Addins** folder as in Figure 1-2.

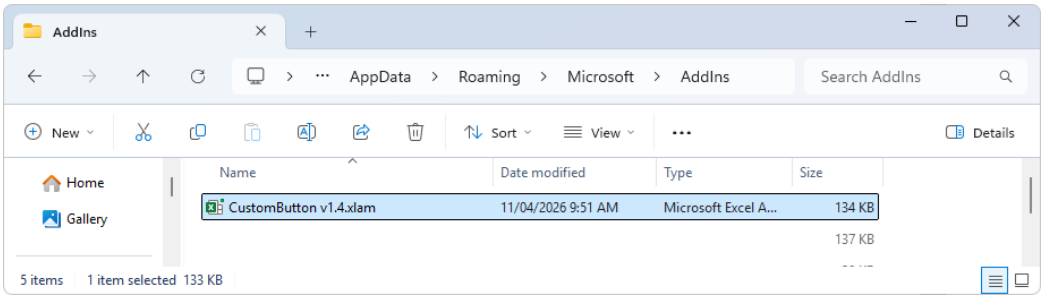

Figure 1-2: Excel Add-ins Folder

1.2.2 Important: Unblocking the Add-in

Because the file was downloaded from an external source (GitHub), Windows will apply a 'Mark of the Web' security attribute, which prevents the Add-in from executing macros. Even if the file is placed in the correct directory, it will not function until it is manually unblocked.

To unblock the Add-in:

1. Navigate to the %appdata%\Microsoft\AddIns folder
2. **Right-click** the CustomButton v1.4.xlam file.
3. Select **Properties**.
4. At the bottom of the **General** tab, look for **Security**.
5. Check the **Unblock** box and click **Apply**.

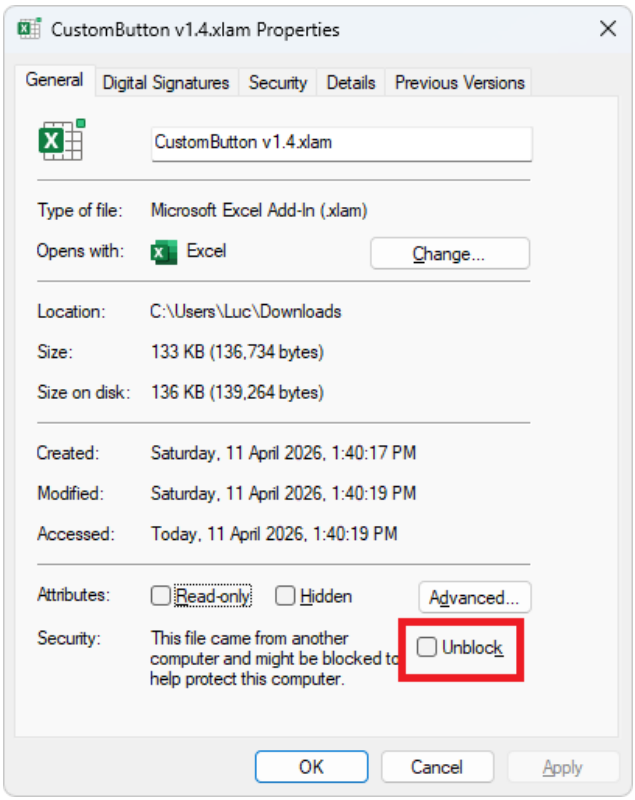

Figure 1-3: Unblock Downloaded CustomButton Add-in

2 Working with CustomButton

Before the **CustomButton** class can be utilized within your UserForms, it must be properly initialized within the VBA environment. This involves linking the Add-in to your specific workbook and ensuring the VBA compiler can access the library's properties and methods.

2.1 Referencing CustomButton in VBA

Once the Add-in is installed and unblocked, you must establish a reference to the library within your specific VBA Project. This allows the VBA compiler to recognize the **CustomButton** class and provide IntelliSense support while coding.

Follow these steps to enable the reference:

1. Open Excel.
2. Go to the **Developer** tab (If you don't see it, right-click any ribbon tab > Customize the Ribbon > Check "Developer").
3. Click **View Code**.
4. Go to the **Tools** menu > **References..** and locate CustomButton_v1_4 and select it as in Figure 2-1. A tick mark will appear next to CustomButton_v1_4.
5. Click **OK**.

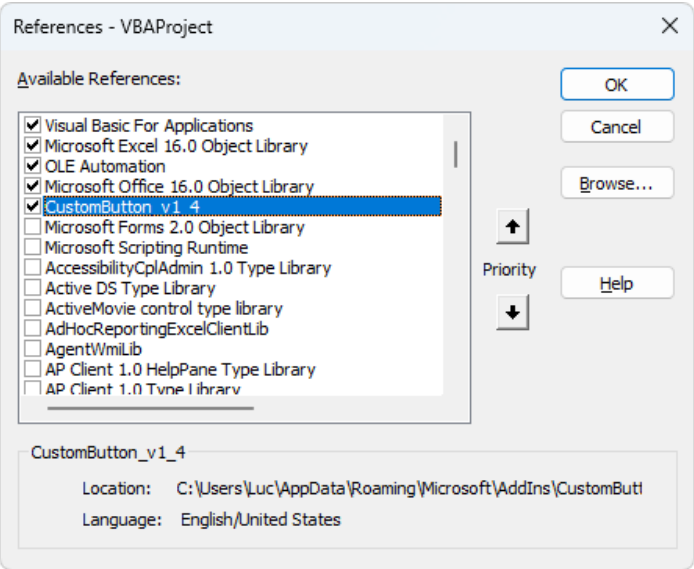

Figure 2-1: Adding CustomButton_v1_4 as a Reference

At this stage the add-in is now available to use in your UserForms.

3 CustomButtons "Quick Start" Guide

Follow the steps below to add a **CustomButton** to your **UserForm**. This process allows you to quickly replace standard legacy CommandButtons with the CustomButton class, providing an immediate visual upgrade to your interface at run time.

3.1 ShowUserFormCode() Function

For existing UserForms, the most efficient method for migrating from legacy controls to **CustomButtons** is utilizing the `ShowUserFormCode()` function. By passing the **`Me`** keyword as a parameter at the conclusion of the `UserForm_Initialize()` event, the function generates the necessary boilerplate code. Upon execution, the Immediate Window displays the specific code required to be merged back into the UserForm module, as illustrated in Figure 3-1. The example below demonstrates the implementation, for the required integration:

```
Private Sub UserForm_Initialize()
    Dim hWnd As LongPtr

    With SampleForm
        .Height = 180#          'fix screen driver issue
        .Width = 240#

        .StartUpPosition = 0    'allow manual positioning

        hWnd = FindWindow("ThunderDFrame", .Caption)
        CentreUserForm hWnd
    End With

    'generates boilerplate code in the Immediate Window
    CustomButtons.ShowUserFormCode Me
End Sub
```

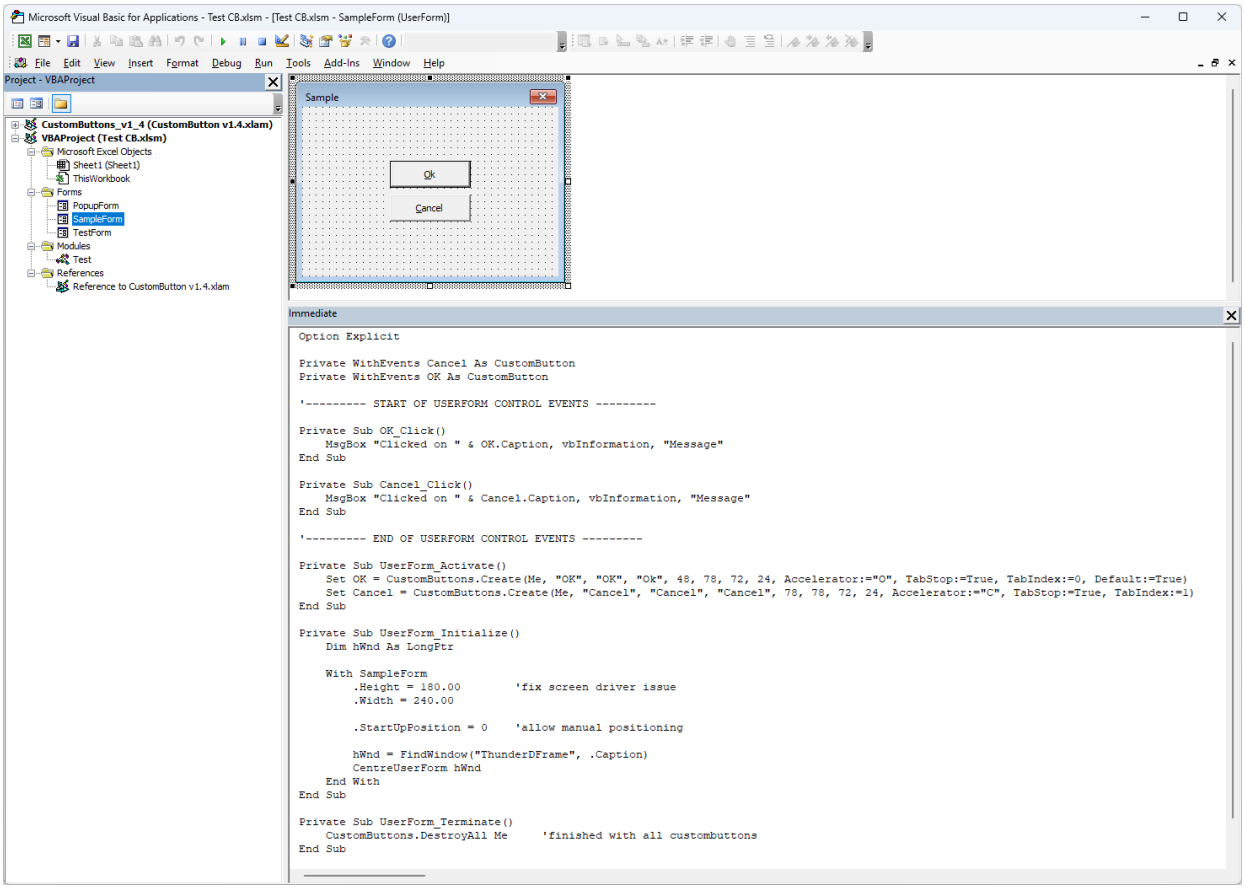

Figure 3-1: ShowUserFormCode() Output

Merge the required code segments from the **Immediate Window** back into your **UserForm** code. After integration is complete, navigate to Section 3.2 for instructions on removing the original CommandButtons.

3.2 Deleting the Legacy CommandButtons

Deleting the original **CommandButtons** is a critical requirement of the migration process. Because the **CustomButton** class dynamically instantiates its own controls using the original naming convention, maintaining duplicate button names will result in a naming conflict. Failure to remove the legacy button prior to execution will trigger a **Runtime Error**.

3.3 _Entry and _Exit Events

Due to native VBA limitations, custom classes cannot directly sink the standard `_Exit` event of a control. To provide this functionality, the **CustomButton** class utilizes the `_OnExit` events instead and for consistency the `_OnEnter event`. If your existing UserForm logic relies on either of these events, you must rename your subroutines to match the class requirements as shown below:

```
Private Sub Button1_OnEnter()
    'Button1 enter code goes here
End Sub

Private Sub Button1_OnExit(ByVal Cancel As MSForms.ReturnBoolean)
    'Button1 exit code goes her
End Sub
```

3.4 FindWindow() Windows API

The `FindWindow()` API function is included as a dedicated utility to retrieve the hWnd (Window Handle) of the UserForm. This handle is used exclusively to calculate the form's precise coordinates on the screen, ensuring the UserForm is correctly centered within the Excel application window.

The implementation utilizes conditional compilation (`PtrSafe` and `LongPtr`) to ensure full compatibility across both **32-bit and 64-bit** Excel environments. By obtaining the hWnd, the **CustomButton** class can bypass standard VBA positioning limitations, providing a more consistent user experience on high-resolution or multi-monitor configurations.

3.5 CentreUserForm() Function

The `CentreUserForm()` function is a layout utility designed to align the UserForm precisely within the boundaries of the main Excel application window. This function requires a single parameter—the hWnd (Window Handle) retrieved in the previous step—to calculate the parent window's coordinates.

Unlike the native VBA StartUpPosition property, which defaults to the center of the physical screen, `CentreUserForm()` ensures the interface remains **contained** within the active Excel workspace. This provides a more cohesive user experience, particularly for users operating on ultra-wide or multi-monitor setups where a screen-centered form may appear disconnected from the application.

4 Functional Specifications

This section provides a technical overview of the public classes, methods, and properties available within the **CustomButton** framework. These components allow for the programmatic initialization, management, and customization of your UserForm interface.

4.1 Class CustomButtons

The CustomButtons class serves as the primary engine for the library. It manages the global state of all button instances and is the core class pre-defined within the **CustomButton** add-in. The following methods are used to initialize and manage the button lifecycle:

Method	Description
Create	Initializes and renders a new button instance on the UserForm.
DestroyAll	Terminates all active button instances and clears them from memory.
ShowUserFormCode	Scans the current UserForm and generates the necessary boilerplate code for integration.

Table 4-1: CustomButtons Class Methods

The following properties allow for global configuration of the button environment:

Property	Note
ExcelGreen	A Boolean property to toggle the signature "Excel Green" theme across all buttons.
Trace	A debugging utility that enables or disables execution logs for troubleshooting.

Table 4-2: CustomButtons Class Properties

4.1.1 .Create() Method

The `.Create()` method is the primary constructor used to instantiate modern buttons within a VBA project. It supports deployment on standard UserForms as well as nested containers (Frames and MultiPage controls).

The method returns a **CustomButton** class object. Because this returns an object, the Set keyword must be used when assigning the result to a variable.

Syntax Example:

```
Set Button1 =
      CustomButtons.Create(Me, "Button1", "Submit1", "Submit", 10, 10)
```

The following table outlines the arguments available for the `.Create()` method:

Parameters	Required	Type	Default Value	Note
Parent	Yes	Object		UserForm, Frame or Multipage handle
VariableName	Yes	String		Declared variable Name
Name	Yes	String		
Caption	Yes	String		
Top	Yes	Single		In points
Left	Yes	Single		In points
Width	-	Single	68	In points
Height	-	Single	20	In points
FontName	-	String	Tahoma	
FontSize	-	Single	8	In points
FontBold	-	Boolean	False	
FontItalic	-	Boolean	False	
FontUnderline	-	Boolean	False	
FontStrikethrough	-	Boolean	False	
Accelerator	-	String		Empty by default
Cancel	-	Boolean	False	ESC triggers a button click
TabIndex	-	Integer	-1	
TabStop	-	Boolean	True	
Tag	-	String		Store custom data string with button
Default	-	Boolean	False	
Visible	-	Boolean	True	
Enabled	-	Boolean	True	

Table 4-3: CustomButtons.Create() Parameters

4.1.2 .DistroyAll() Method

The `.DestroyAll()` method is used to terminate all active button instances within a specific container and release them from memory. This is particularly useful when a UserForm is being closed or when a container (such as a Frame) needs to be cleared and re-populated dynamically.

By passing the parent object as a parameter, the method identifies all associated **CustomButton** objects and ensures they are safely disposed of, preventing orphaned objects from lingering in memory.

Syntax Example

```
Private Sub UserForm_Terminate()
    'clean up all custom buttons when the form closes
    CustomButtons.DestroyAll Me
End Sub
```

The following table outlines the arguments required for the `.DestroyAll()` method:

Parameter	Required	Type	Note
Parent	Yes	Object	Handle from a UserForm, Frame or Multipage. Often will be the keyword `Me.`

Table 4-4: CustomButtons.DistoryAll() Parameters

4.1.3 .ShowUserFormCode() Method

The `.ShowUserFormCode()` method is a productivity utility designed to simplify the integration of CustomButton logic into your project. When called, it scans the specified parent container and generates a ready-to-use block of VBA code in the Immediate Window.

This generated code includes the necessary variable declarations and event handlers (e.g., Click, Hover, and MouseMove) for every button identified within the container. This eliminates the need for manual coding and ensures that all event signatures are technically correct. See Section 3.1 for an usage example.

The following table outlines the arguments required for the `.ShowUserFormCode()` method:

Parameter	Required	Type	Note
Parent	Yes	Object	Handle from a UserForm, Frame or Multipage. Often will be the keyword `Me.`

Table 4-5: CustomButtons.ShowUserFormCode() Parameters

4.1.4 .ExcelGreen Property

The `.ExcelGreen` property is a global Boolean flag that determines the default color scheme for all button instances created after the property is set. When enabled, buttons are rendered using a professional palette inspired by the Microsoft Excel branding, providing a seamless "native" look for your add-in.

Setting this property to False (default) allows buttons to use standard minimalist styling or custom colors defined at the individual button level.

Usage Example

```
'apply Excel branding globally before creating buttons
CustomButtons.ExcelGreen = True
```

The following table describes the impact of the `.ExcelGreen` setting:

Property	Value
ExcelGreen	True will applies a specific color palette to match Excel green colors. False will revert back to either Windows 10 or 11 colors. See Table 4-10.

Table 4-6: CustomButtons.ExcelGreen Porperty

4.1.5 .Trace Property

The `.Trace` property is a diagnostic utility used to output real-time execution data to the VBA Immediate Window. By assigning specific constants to this property, developers can isolate and troubleshoot specific behaviors—such as event triggers, layout rendering, or object creation—without wading through unnecessary log data.

This property supports bitwise-style constants, allowing for high-precision debugging during the development and testing phases of your UserForm.

Syntax Example

```
' Enable logging for events and creation only
CustomButtons.Trace = TR_EVENTS + TR_CREATE
```

The following constants can be assigned to the `.Trace` property to define the logging level:

Constant	Description
TR_NONE	Disables all diagnostic logging (Default).
TR_EVENTS	Logs interaction events (Click, DblClick, Enter, Exit).
TR_METHODS	Logs calls to internal and public methods.
TR_PROPERTIES	Logs changes to button property values.
TR_LAYOUTCOLORS	Logs rendering data related to themes and color schemes.
TR_USERFORM	Logs interactions specifically related to the host UserForm.
TR_MOUSEMOVE	Logs high-frequency mouse movement data.
TR_CREATE	Logs data during the instantiation and initialization phase.
TR_DESTROYALL	Logs the disposal and memory release process.
TR_OVERRIDEDEFAULT	Logs instances where default behaviors are bypassed.
TR_ALL	Enables all available logging levels simultaneously.

Table 4-7: CustomButtons.Tracen Porperty

4.2 Class CustomButton

The CustomButton class represents an individual button instance created via the `CustomButtons.Create()` method. This class acts as a sophisticated bridge between the user's mouse/keyboard actions and the UserForm, managing internal state while exposing standard events and properties to the developer.

4.2.1 Event Support

The class listens for and passes the following events through to your project. This allows **CustomButtons** to behave identically to native controls while utilizing a modern render engine.

1. Click / DblClick
2. MouseDown / MouseUp / MouseMove
3. KeyPress / KeyDown / KeyUp
4. OnEnter / OnExit (Focus events)
5. BeforeUpdate / AfterUpdate

4.2.2 Methods

Use these methods to programmatically control a specific button instance:

Method	Description
Repaint	Forces the button to re-render its visual state (useful after changing colors or captions).
SetFocus	Programmatically moves the keyboard focus to the button.

Table 4-8: CustomButton Methods

4.2.3 Properties

These properties allow you to read or modify the button's state at runtime.

Property	Type	Access	Note
.Parent	Object	Read-Only	Returns the container (UserForm/Frame) holding the button.
.VarName	String	Read-Only	Returns the name assigned to the variable in VBA.
.Name	String	Read-Only	Returns the internal name of the control.
.Caption	String	Read/Write	The text displayed on the button surface.
.Top	Single	Read/Write	The position of the button top in points.
.Left	Single	Read/Write	The position of the button left in points.
.Width	Single	Read/Write	The dimensions of the button width in points.
.Height	Single	Read/Write	The dimensions of the button height in points.
.Font	Object	Read-Only	Access to the font sub-object.
.Font.Name	String	Read/Write	Recommended: 'Segoe UI' for modern OS feel.
.Font.Size	Single	Read/Write	Recommended: 8.5 for high-density UIs.
.Accelerator	String	Read/Write	The hotkey (Alt + Key) for the button.
.Cancel	Boolean	Read/Write	Set to True to map the button to the Esc key.
.Default	Boolean	Read/Write	Set to True to map the button to the Enter key.
.TabIndex	Integer	Read/Write	The order of the button in the keyboard Tab sequence.
.TabStop	Boolean	Read/Write	Determines if the button can receive focus via the Tab key.
.Tag	String	Read/Write	A storage string for developer-defined metadata.
.Visible	Boolean	Read/Write	Toggles the button's visibility.
.Enabled	Boolean	Read/Write	Toggles user interactivity and "greyed-out" state.

Table 4-9: CustomButton Methods

4.2.4 Style & Color Properties

The CustomButton class allows for granular control over the control's visual states (Normal, Hover, Pressed, and Disabled). To assist with UI consistency, the following table provides the default RGB values for achieving a "Native" look across different environments.

Property (Color)	Windows 10 (Classic)	Windows 11 (Fluent)	Excel (Signature Green)
.BackColor	RGB(225, 225, 225)	RGB(251, 251, 251)	—
.BorderColor	RGB(173, 173, 173)	RGB(229, 229, 229)	—
.ForeColor	RGB(0, 0, 0)	RGB(0, 0, 0)	—
.HoverBackColor	RGB(229, 241, 251)	RGB(224, 238, 249)	RGB(233,245,238)
.HoverDownColor	RGB(204, 228, 247)	RGB(204, 228, 247)	RGB(159,205,179)
.HoverBorderColor	RGB(0, 120, 215)	RGB(0, 120, 212)	RGB(78,150,104)
.HoverForeColor	RGB(0, 0, 0)	RGB(0, 0, 0)	—
.DisabledBackColor	RGB(204, 204, 204)	RGB(249, 249, 249)	—
.DisabledBorderColor	RGB(191, 191, 191)	RGB(233, 233, 233)	—
.DisabledForeColor	RGB(131, 131, 131)	RGB(158, 158, 158)	—
.DefaultBorderColor	RGB(0, 120, 215)	RGB(0, 103, 192)	RGB(78,150,104)

Table 4-10: Default Color Schemes by Operating System / Application